1. JavaScript was originally developed by Brendan Eich at Netscape under the code name:

(A) Mocha
(B) LiveWire
(C) Oak
(D) CoffeeScript

2. The official standardized language specification governing JavaScript is:

(A) ECMAScript (ECMA-262)
(B) W3C Script
(C) ISO/IEC JavaScript
(D) IEEE 802.11 Script

3. Which HTML tag is used to embed or reference client-side JavaScript code?

(A) <script>
(B) <javascript>
(C) <code>
(D) <js>

4. In JavaScript, which modern keyword declares a block-scoped reassignable variable?

(A) let
(B) var
(C) const
(D) def

5. In JavaScript, which keyword declares a block-scoped read-only constant reference?

(A) const
(B) let
(C) var
(D) static

6. Which legacy JavaScript keyword declares a function-scoped or globally scoped variable subject to hoisting?

(A) var
(B) let
(C) const
(D) val

7. Which built-in JavaScript function outputs diagnostic information directly to the browser debugging console?

(A) console.log()
(B) print()
(C) document.write()
(D) system.out()

8. Which pop-up dialog function displays an informational alert box with an OK button?

(A) alert()
(B) confirm()
(C) prompt()
(D) message()

9. Which pop-up dialog function displays a modal box with OK and Cancel buttons, returning a boolean?

(A) confirm()
(B) alert()
(C) prompt()
(D) dialog()

10. Which pop-up dialog function prompts the user to enter text input, returning the entered string or null?

(A) prompt()
(B) input()
(C) confirm()
(D) alert()

11. What are the seven primitive data types in modern JavaScript (ES6+)?

(A) Number, String, Boolean, Undefined, Null, Symbol, BigInt
(B) Integer, Float, String, Boolean, Array, Object
(C) Number, Text, Boolean, Nil, Void
(D) Char, String, Int, Double, Long

12. What does the unary 'typeof' operator return for an undeclared variable or a variable without an assigned value?

(A) "undefined"
(B) "null"
(C) "empty"
(D) "void"

13. What legacy bug in JavaScript causes 'typeof null' to return an unexpected result?

(A) It returns "object"
(B) It returns "null"
(C) It returns "undefined"
(D) It throws a TypeError

14. Which comparison operator checks for value equality WITH automatic type coercion?

(A) == (abstract equality)
(B) === (strict equality)
(C) =
(D) !==

15. Which comparison operator checks for strict equality WITHOUT type coercion (both value and type must match)?

(A) === (strict equality)
(B) ==
(C) !=
(D) equals()

16. What is the result of evaluating '5' == 5 in JavaScript?

(A) true (string '5' is coerced to number 5)
(B) false
(C) NaN
(D) TypeError

17. What is the result of evaluating '5' === 5 in JavaScript?

(A) false (types String and Number do not match)
(B) true
(C) null
(D) undefined

18. Which special numeric value represents a computational error resulting from an undefined or unrepresentable mathematical operation?

(A) NaN (Not-a-Number)
(B) null
(C) undefined
(D) Infinity

19. Which global function is used to reliably test whether a value is NaN?

(A) Number.isNaN()
(B) isNotNumber()
(C) typeof value === 'NaN'
(D) value == NaN

20. What is the result of the expression NaN === NaN in JavaScript?

(A) false (NaN is the only value in JavaScript not equal to itself)
(B) true
(C) null
(D) undefined

21. Which control flow statement executes a block of code conditionally based on a boolean expression?

(A) if...else
(B) for
(C) while
(D) try...catch

22. Which statement allows multi-way branching by evaluating an expression against multiple matching cases?

(A) switch...case
(B) select...case
(C) choose
(D) branch

23. Which keyword is required inside a switch case to prevent automatic fallthrough into subsequent cases?

(A) break
(B) exit
(C) stop
(D) continue

24. Which loop executes a block of code as long as a specified condition evaluates to true?

(A) while loop
(B) for loop
(C) do...while loop
(D) foreach loop

25. Which loop guarantees that its body will execute at least once before evaluating the condition?

(A) do...while loop
(B) while loop
(C) for loop
(D) for...in loop

26. Which loop is standardly used for iterating with an initialization, condition, and increment counter?

(A) for loop
(B) while loop
(C) do...while loop
(D) switch loop

27. Which loop specifically iterates over the enumerable property keys of an object?

(A) for...in loop
(B) for...of loop
(C) forEach()
(D) while

28. Which ES6 loop iterates over the iterable values of an array, string, set, or map?

(A) for...of loop
(B) for...in loop
(C) while
(D) for...each

29. How is a traditional named function declared in JavaScript?

(A) function greet(name) { ... }
(B) def greet(name):
(C) fn greet(name) { ... }
(D) void greet(name) { ... }

30. What is an anonymous function in JavaScript?

(A) A function created without a designated function name
(B) A function that has no parameters
(C) A function that returns nothing
(D) A private class method

31. What syntax introduced in ES6 provides a concise way to write functions using the '=>' operator?

(A) Arrow Function syntax
(B) Lambda syntax
(C) Pointer function
(D) Inline closure

32. How do arrow functions handle the 'this' keyword differently from standard function declarations?

(A) Arrow functions lexically bind 'this' from the enclosing lexical scope; standard functions bind 'this' dynamically based on how they are called
(B) Arrow functions have no 'this' and throw an error
(C) Arrow functions bind 'this' to the window always
(D) Arrow functions make 'this' immutable

33. An array in JavaScript is created using which bracket notation?

(A) Square brackets [ ]
(B) Curly braces { }
(C) Parentheses ( )
(D) Angle brackets < >

34. Which array method appends one or more elements to the end of an array and returns the new length?

(A) push()
(B) pop()
(C) shift()
(D) unshift()

35. Which array method removes the last element from an array and returns that element?

(A) pop()
(B) push()
(C) shift()
(D) slice()

36. Which array method removes the first element from an array and shifts all remaining elements down?

(A) shift()
(B) unshift()
(C) pop()
(D) splice()

37. Which array method adds one or more elements to the beginning of an array?

(A) unshift()
(B) shift()
(C) push()
(D) prepend()

38. Which array method returns a shallow copy of a portion of an array without modifying the original array?

(A) slice()
(B) splice()
(C) split()
(D) extract()

39. Which array method changes the contents of an array in place by removing, replacing, or adding elements?

(A) splice()
(B) slice()
(C) concat()
(D) modify()

40. Which method converts a JavaScript object or value into a JSON formatted string?

(A) JSON.stringify()
(B) JSON.parse()
(C) JSON.toString()
(D) JSON.encode()

41. What is 'Hoisting' in JavaScript execution?

(A) Variable and function declarations are moved to the top of their containing scope during the compilation phase before execution
(B) Functions are executed before variables are read
(C) Variables are converted to objects
(D) Memory is garbage collected automatically

42. How does hoisting differ between 'var' and 'let'/'const'?

(A) 'var' is hoisted and initialized with undefined; 'let' and 'const' are hoisted but remain uninitialized in the Temporal Dead Zone (TDZ)
(B) 'let' and 'const' are not hoisted at all
(C) 'var' throws a ReferenceError if used early
(D) There is zero difference

43. What error is thrown when accessing a 'let' variable within its scope prior to its declaration statement?

(A) ReferenceError (due to the Temporal Dead Zone)
(B) TypeError
(C) SyntaxError
(D) UndefinedError

44. What is a Closure in JavaScript?

(A) A function bundled together with references to its surrounding lexical state (lexical environment), allowing it to access outer variables even after the outer function has closed
(B) A function that terminates the program
(C) A method to seal an object
(D) A private class constructor

45. Which functional array method creates a new array populated with the results of calling a provided function on every element?

(A) map()
(B) filter()
(C) forEach()
(D) reduce()

46. Which functional array method creates a shallow copy of a portion of an array containing only elements that pass a test condition?

(A) filter()
(B) map()
(C) find()
(D) some()

47. Which array method executes a user-supplied reducer callback function on each element of the array, resulting in a single output value?

(A) reduce()
(B) accumulate()
(C) map()
(D) aggregate()

48. What does the array method 'some()' return?

(A) true if at least one element in the array passes the callback test; otherwise false
(B) An array of matching items
(C) The index of the first match
(D) The total count of matches

49. What does the array method 'every()' return?

(A) true if every single element in the array passes the callback test; otherwise false
(B) true if array is not empty
(C) All matching items
(D) A copy of the array

50. What is the output of [1, 2, 3].reduce((acc, curr) => acc + curr, 10)?

(A) 16 (initial accumulator 10 + 1 + 2 + 3)
(B) 6
(C) 10
(D) 15

51. What is the difference between shallow copying and deep copying of an object in JavaScript?

(A) Shallow copy copies top-level properties but copies references for nested objects; deep copy recursively copies all nested levels independently
(B) Shallow copy converts to JSON; deep copy does not
(C) Shallow copy is immutable; deep copy is mutable
(D) There is no difference in JavaScript

52. Which method introduced in modern JavaScript (ES2022) natively creates a deep copy of complex objects and nested data structures?

(A) structuredClone()
(B) Object.assign()
(C) JSON.parse(JSON.stringify())
(D) Object.clone()

53. What limitation exists when using JSON.parse(JSON.stringify(obj)) to clone an object?

(A) It discards functions, undefined, Symbols, RegExps, Date objects are converted to strings, and it throws an error on circular references
(B) It only works on arrays
(C) It converts numbers to strings
(D) It is 100 times slower than any loop

54. How does prototypal inheritance work in JavaScript?

(A) Every JavaScript object has an internal hidden link to another object called its prototype, from which it inherits properties and methods
(B) Classes compile into bytecode interfaces
(C) Objects inherit through multiple inheritance tables
(D) Prototypes are copied into RAM for each instance

55. Which property or method allows inspecting the prototype of an object instance in modern JavaScript?

(A) Object.getPrototypeOf(obj) (or __proto__)
(B) obj.prototype
(C) obj.getSuper()
(D) Object.prototypeOf(obj)

56. What is the prototype chain?

(A) A sequence of linked prototype objects inspected sequentially when a property lookup does not find the property on the object itself, terminating at Object.prototype (null)
(B) A circular array of functions
(C) The call stack of active functions
(D) A queue of event handlers

57. What does the 'new' keyword execute when calling a constructor function `new Person('Alice')`?

(A) Creates a blank object, sets its prototype to Person.prototype, binds 'this' to the new object, executes the function, and returns the object
(B) Allocates heap memory and calls free()
(C) Clones the window object
(D) Converts function to JSON

58. What is the difference between Function Declaration and Function Expression in JavaScript regarding hoisting?

(A) Function Declarations are fully hoisted with their implementation; Function Expressions assigned to variables are not hoisted (or hoisted as undefined if with var)
(B) Function Expressions are hoisted; Declarations are not
(C) Both can be called before declaration identically
(D) Neither is hoisted in strict mode

59. What is an Immediately Invoked Function Expression (IIFE)?

(A) A function expression that is executed immediately upon definition: (function() { ... })();
(B) A function that executes inside a Web Worker
(C) A callback passed to setTimeout()
(D) A constructor function invoked with new

60. Why were IIFEs ubiquitous in legacy JavaScript prior to ES6 Modules?

(A) To create a local private scope and avoid polluting the global namespace with temporary variables
(B) To speed up loop performance
(C) To prevent syntax errors
(D) To handle asynchronous AJAX calls

61. What does the JavaScript 'strict mode' ('use strict';) enforce?

(A) Eliminates silent errors by throwing exceptions, disables 'with', prevents assigning to undeclared global variables, and secures 'this' binding
(B) Restricts variable types strictly to TypeScript types
(C) Compiles code to WebAssembly
(D) Disables all prototype inheritance

62. What is the output of the expression `0.1 + 0.2 === 0.3` in JavaScript?

(A) false (due to IEEE 754 binary floating-point rounding: 0.1 + 0.2 evaluates to 0.30000000000000004)
(B) true
(C) NaN
(D) TypeError

63. How should floating-point numbers be accurately compared for equality in JavaScript?

(A) Math.abs(a - b) < Number.EPSILON
(B) a == b
(C) a === b
(D) Number.isEqual(a, b)

64. What are the six falsy values in JavaScript boolean coercion?

(A) false, 0 (and -0, 0n), "" (empty string), null, undefined, and NaN
(B) false, 0, "0", null, undefined, NaN
(C) false, null, undefined, [], {}
(D) false, 0, "false", null

65. What is the result of evaluating Boolean([]) and Boolean({}) in JavaScript?

(A) true and true (all objects and arrays are truthy in JavaScript, even if empty)
(B) false and false
(C) true and false
(D) false and true

66. What does the Logical OR (||) operator return in JavaScript expressions like `a || b`?

(A) Returns the first truthy operand, or the last operand if all are falsy
(B) Always returns true or false boolean
(C) Returns true if both are truthy
(D) Throws an error if a is falsy

67. What does the Logical AND (&&) operator return in expressions like `a && b`?

(A) Returns the first falsy operand, or the last operand if all are truthy
(B) Always returns true or false
(C) Returns the sum of operands
(D) Returns null if either is falsy

68. What does the Nullish Coalescing operator (??) do in `a ?? b`?

(A) Returns the right-hand operand 'b' ONLY if the left-hand operand 'a' is null or undefined (preserves 0 and false)
(B) Returns b if a is falsy
(C) Returns true if both are null
(D) Performs bitwise comparison

69. How does `a ?? b` differ from `a || b` when a = 0 or a = ""?

(A) `a ?? b` evaluates to 0 or "" (retaining the value); `a || b` treats 0 and "" as falsy and incorrectly falls back to b
(B) `a ?? b` throws an exception
(C) They behave identically
(D) `a || b` returns boolean true

70. What does the Optional Chaining operator (?.) achieve in `user?.profile?.address?.city`?

(A) Safely reads deeply nested properties; if any reference in the chain is null or undefined, evaluation short-circuits and returns undefined instead of throwing a TypeError
(B) Creates the missing properties automatically
(C) Throws a custom error
(D) Transforms the object to a string

71. Which method parses a JSON string, constructing the JavaScript value or object described by the string?

(A) JSON.parse()
(B) JSON.stringify()
(C) JSON.decode()
(D) JSON.objectify()

72. What error is thrown if JSON.parse() receives an invalid JSON string (e.g., single quotes or trailing commas)?

(A) SyntaxError
(B) TypeError
(C) ReferenceError
(D) RangeError

73. Which built-in JavaScript object provides mathematical constants and functions like Math.PI, Math.round, and Math.sqrt?

(A) Math
(B) Number
(C) Calculate
(D) Geometry

74. How do you generate a pseudo-random integer between min and max (inclusive) in JavaScript?

(A) Math.floor(Math.random() * (max - min + 1)) + min
(B) Math.random(min, max)
(C) Math.round(Math.random() * max)
(D) Random.next(min, max)

75. Which built-in object represents a single moment in time in a platform-independent format measured in milliseconds since Unix Epoch?

(A) Date
(B) Time
(C) Timestamp
(D) Clock

76. What is the output of the classic closure loop: `for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 100); }`?

(A) Logs 3, 3, 3 (because 'var' is function-scoped; all three callbacks share the single mutated variable i after loop termination)
(B) Logs 0, 1, 2
(C) Logs 1, 2, 3
(D) Logs undefined, undefined, undefined

77. How does changing 'var i = 0' to 'let i = 0' solve the issue in Q76 and output 0, 1, 2?

(A) 'let' creates a new lexical block scope binding for 'i' on each individual loop iteration, preserving each distinct value in closure
(B) 'let' stops the loop immediately
(C) 'let' makes setTimeout synchronous
(D) 'let' hoists the loop to global scope

78. What are the four explicit rules governing how the 'this' keyword is bound in JavaScript functions?

(A) 1. new binding (constructor); 2. Explicit binding (call/apply/bind); 3. Implicit binding (context object); 4. Default binding (global / undefined in strict mode)
(B) 1. Class binding; 2. Global binding; 3. Arrow binding; 4. Local binding
(C) 1. Lexical; 2. Prototype; 3. Variable; 4. Constant
(D) 1. HTML tag; 2. Event listener; 3. Window; 4. Document

79. What is the difference between Function.prototype.call() and Function.prototype.apply()?

(A) call() accepts function arguments individually as a comma-separated list; apply() accepts arguments as a single array
(B) call() is asynchronous; apply() is synchronous
(C) call() binds permanently; apply() binds temporarily
(D) apply() only works in strict mode

80. What does Function.prototype.bind() return when invoked on a function?

(A) A new copy of the target function with its 'this' context permanently bound to the provided object, regardless of how it is later called
(B) The return value of the executed function immediately
(C) A Promise that resolves with this
(D) An arrow function clone

81. What happens when you use 'bind()' or 'call()' on an Arrow Function?

(A) The binding is silently ignored; arrow functions retain their lexical 'this' binding from definition time and cannot be rebound
(B) It throws a TypeError
(C) It changes this for that single invocation
(D) It transforms the arrow function into a regular function

82. What is a JavaScript Generator Function (function*), and what does the 'yield' keyword do?

(A) A function that can pause its execution midway using 'yield' and resume later when its iterator's next() method is called
(B) A function that generates random numbers
(C) A recursive constructor
(D) A function that runs in a Web Worker

83. What does calling `generator.next()` on a generator instance return?

(A) An object with two properties: { value: Any, done: Boolean }
(B) The yielded value directly
(C) A Promise
(D) The generator function pointer

84. What is the JavaScript Event Loop, and how does it coordinate the Call Stack, Web APIs, Task Queue (Macrotasks), and Microtask Queue?

(A) The event loop continuously monitors the Call Stack; when empty, it drains all Microtasks (Promises), then picks the oldest Macrotask (setTimeout)
(B) It executes functions on multiple CPU cores simultaneously
(C) It converts JavaScript into WebAssembly
(D) It loops through all DOM elements checking for clicks

85. Between Promise callbacks (.then / queueMicrotask) and setTimeout(fn, 0), which executes first after the synchronous call stack empties?

(A) The Promise callback executes first because Microtasks have strict priority and must be completely exhausted before Macrotasks run
(B) setTimeout executes first
(C) They execute in random order
(D) They run concurrently in parallel

86. What is the output order of: `console.log(1); setTimeout(() => console.log(2), 0); Promise.resolve().then(() => console.log(3)); console.log(4);`?

(A) 1, 4, 3, 2 (1 and 4 are synchronous; 3 is Microtask; 2 is Macrotask)
(B) 1, 2, 3, 4
(C) 1, 3, 4, 2
(D) 1, 4, 2, 3

87. What is the purpose of the JavaScript Proxy object (new Proxy(target, handler))?

(A) Enables custom meta-programming behavior for fundamental operations on a target object (e.g., property lookup, assignment, enumeration, invocation)
(B) Acts as an HTTP network proxy for Fetch requests
(C) Creates an encrypted SSL tunnel
(D) Caches array elements in memory

88. What is the Reflect API in JavaScript, and how does it complement the Proxy object?

(A) Provides static methods for interceptable JavaScript operations, allowing default internal behaviors to be forwarded cleanly inside Proxy traps
(B) Reflects HTML DOM elements on canvas
(C) Inspects private class fields via reflection
(D) Inverts matrix arrays

89. What is a JavaScript Symbol, and why is it used as an object property key?

(A) A unique, immutable primitive data type guaranteed never to collide with any other property key, ideal for creating private or non-enumerable object metadata
(B) A string representation of a function
(C) A mathematical operator
(D) A cryptographic hash token

90. What does the Well-Known Symbol `Symbol.iterator` specify on an object?

(A) Specifies the default iterator method for an object, making it iterable via for...of loops and the spread operator [...obj]
(B) Marks the object as a generator
(C) Prevents object mutation
(D) Enables JSON serialization

91. What is the difference between Map and a standard JavaScript Object for key-value storage?

(A) Map accepts any data type as keys (including objects and functions), maintains insertion order, and provides direct size property; Objects only allow strings/symbols as keys
(B) Map is immutable; Object is mutable
(C) Objects cannot store strings
(D) Map does not support iteration

92. What is a WeakMap in JavaScript, and how does it prevent memory leaks?

(A) A collection of key-value pairs where keys MUST be objects and are held as WEAK references, allowing them to be garbage-collected if no other references exist
(B) A Map with a maximum size of 10 items
(C) A Map that clears every 60 seconds
(D) A Map accessible only in strict mode

93. What is a WeakSet in JavaScript?

(A) An unordered collection of unique objects held weakly, used to tag or track objects without preventing their garbage collection
(B) A set of primitive numbers
(C) A set with duplicate values allowed
(D) An immutable array

94. What does `Object.freeze(obj)` do to an object compared to `Object.seal(obj)`?

(A) freeze() makes all existing properties non-writable and non-configurable (completely immutable); seal() prevents adding/deleting properties but allows modifying existing writable properties
(B) seal() is deeper than freeze()
(C) freeze() converts object to JSON
(D) seal() encrypts object properties

95. Is `Object.freeze()` shallow or deep by default in JavaScript?

(A) Shallow: nested child objects are NOT frozen and remain fully mutable unless recursively frozen
(B) Deep: all nested child objects are automatically frozen
(C) It only freezes methods
(D) It only freezes arrays

96. What is the difference between `Object.defineProperty()` and normal assignment `obj.prop = val`?

(A) defineProperty() allows explicit configuration of property attributes: writable, enumerable, and configurable (and getters/setters); normal assignment defaults all to true
(B) defineProperty() is 10 times faster
(C) normal assignment throws an error on objects
(D) defineProperty() cannot set values

97. What is the purpose of JavaScript Tagged Template Literals (e.g., `styled.div\`color: red;\``)?

(A) Allows parsing template literals with a custom function, receiving an array of raw strings and evaluated expression arguments (used in styled-components and SQL escaping)
(B) Formats dates to ISO strings
(C) Encrypts strings with AES
(D) Converts HTML strings to DOM nodes

98. What does the method `Object.preventExtensions(obj)` do?

(A) Prevents new properties from ever being added to the object, though existing properties can still be modified or deleted
(B) Freezes all existing properties
(C) Hides the object from for...in
(D) Deletes all properties

99. What happens if code in 'strict mode' attempts to assign a value to a non-writable property of a frozen object?

(A) It throws a TypeError exception (in non-strict mode, it fails silently without throwing an error)
(B) It overwrites the property anyway
(C) It restarts the browser thread
(D) It logs a warning and continues

100. What is the difference between Tail Call Optimization (TCO) in ES6 and normal recursive function execution?

(A) TCO reuses the current stack frame for recursive calls that occur in the tail position, executing infinite recursion in constant O(1) stack space without stack overflow
(B) TCO converts recursion to while loops at runtime
(C) TCO runs recursion on the GPU
(D) TCO limits recursion depth to 1000

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain JavaScript variable scoping paradigms, comparing var, let, and const in terms of scope, hoisting, and re-declaration.

Q2. Analyze the Temporal Dead Zone (TDZ) in JavaScript, explaining why it exists and illustrating its behavior with code.

Q3. Explain JavaScript Data Types, detailing Primitive Types vs Reference Types and the memory model (Stack vs Heap).

Q4. Explain JavaScript Closures in depth: definition, lexical scoping, memory retention, and practical use cases.

Q5. Analyze Prototypal Inheritance in JavaScript, explaining the prototype chain, Object.prototype, and constructor functions.

Q6. Explain the 'this' keyword in JavaScript, detailing Default, Implicit, Explicit (call/apply/bind), and Lexical (Arrow) binding rules.

Q7. Explain the JavaScript Event Loop, Call Stack, Web APIs, Task Queue (Macrotasks), and Microtask Queue with a sequence diagram description.

Q8. Compare Higher-Order Array Methods in JavaScript: map(), filter(), reduce(), and forEach(), writing functional pipelines.

Q9. Explain JavaScript Object Immutability techniques: Object.preventExtensions(), Object.seal(), and Object.freeze().

Q10. Describe JavaScript Property Descriptors, explaining writable, enumerable, configurable, getters, and setters.

Q11. Explain JavaScript ES6 Classes, constructor methods, static methods, class inheritance (extends, super), and private fields (#).

Q12. Explain JavaScript Generators (function*) and Iterators, detailing the Iterator Protocol and Symbol.iterator.

Q13. Describe JavaScript Proxy and Reflect APIs, illustrating how to implement reactive data-binding and input validation traps.

Q14. Explain JavaScript Map, Set, WeakMap, and WeakSet collections, analyzing algorithmic complexity and garbage collection.

Q15. Explain the JavaScript Garbage Collection process, Mark-and-Sweep algorithm, and common memory leak patterns.

Q16. Analyze JavaScript Type Coercion rules, distinguishing between Explicit vs Implicit coercion in operators (+, -, ==).

Q17. Detail JavaScript Error Handling architecture: try, catch, finally, custom Error classes, and Error causation.

Q18. Explain JavaScript Internationalization API (Intl), detailing currency formatting, relative time, and date localization.

Q19. Describe JavaScript Function Currying and Partial Application, explaining their theoretical foundations in Lambda Calculus.

Q20. Explain JavaScript WebAssembly (Wasm) integration, memory sharing, and its coexistence with the JavaScript runtime.

Q21. Write a robust, recursive JavaScript function to perform a Deep Clone of any arbitrary object, handling nested objects, arrays, and Date objects.

Q22. Write a custom implementation of the higher-order array method Array.prototype.myReduce() conforming to the ECMAScript specification.

Q23. Implement a custom Debounce and Throttle function in vanilla JavaScript, explaining when to deploy each in frontend applications.

Q24. Explain JavaScript Module System evolution: CommonJS (require/module.exports) vs ES6 Modules (import/export).

Q25. Demonstrate how to implement the Publish-Subscribe (Pub/Sub) design pattern in pure JavaScript.

Q26. Explain JavaScript Memory Leaks caused by Closures, detailing why outer variables remain pinned in memory.

Q27. Explain the JavaScript Function.prototype.bind() method and write a custom polyfill myBind().

Q28. Discuss JavaScript Strict Mode ('use strict';), detailing eight specific bugs or risky behaviors it eliminates.

Q29. Explain JavaScript Symbols, detailing Symbol.for(), Symbol.keyFor(), and Symbol as non-enumerable object metadata.

Q30. Analyze the performance characteristics of V8 JavaScript Engine: Ignition Bytecode Interpreter and TurboFan Optimizing Compiler.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

```

Even `typeof b` inside the TDZ throws a ReferenceError, breaking the historical assumption that `typeof` is always 100% safe.

---



**>>Solution Question 6:** Main JavaScript Data Types in detail: Primitive Types vs Reference Types and the memory model (Stack vs Heap).
```

Practical Use Cases:

1. Data Encapsulation & Private Variables (emulating private class fields).
2. Function Currying and Partial Application.
3. Event Handlers and Callback functions that preserve state across asynchronous boundaries.

---

object instance.

Arrow Functions Exception (Lexical this):
Arrow functions have NO `this` of their own. They lexically resolve `this` from the enclosing outer scope at definition time. They cannot be rebound using call, apply, or bind.

```
{ name: 'Phone', price: 500, inStock: false },
{ name: 'Tablet', price: 300, inStock: true }
];

// Calculate total cost of in-stock items with 10% tax:
const total = products
.filter(p => p.inStock)
.map(p => p.price * 1.10)
.reduce((sum, current) => sum + current, 0);

console.log(total); // (1000*1.1) + (300*1.1) = 1100 + 330 = 1430
```

---

4. configurable: Boolean indicating whether property attributes can be modified and whether the property can be deleted from the object via `delete`.

Accessor Descriptors (Getters & Setters):
```javascript
const user = { firstName: 'Ada', lastName: 'Lovelace' };
Object.defineProperty(user, 'fullName', {
get() { return `${this.firstName} ${this.lastName}`; },
set(val) { [this.firstName, this.lastName] = val.split(' '); },
enumerable: true,
configurable: true
});
```

>> Solution to Question 11: Explain JavaScript ES6 Classes, constructor methods, static methods, class inheritance (extends, super), and private fields (#).

>>Solution Question 18: Java Script Operators Reflect API Illustrations: Deletion Interactive Data Structures validation traps.

```
}
};

const person = new Proxy({}, validator);
person.age = 25; // Valid
// person.age = -5; // Throws TypeError: Age must be a positive integer.
```

Modern frontend frameworks (Vue 3, MobX) use Proxy traps to trigger automatic UI re-renders whenever state changes.

>> Solution Question 5: Explain JavaScript Map, Set, WeakMap, and WeakSet. When would you choose a Map over a plain object? How do WeakMaps help prevent memory leaks in caching scenarios?

2. Forgotten Timers / Callbacks: `setInterval` running indefinitely retaining references to outer objects in closure.
3. Detached DOM Nodes: JavaScript variables holding references to DOM elements removed from the page.
4. Unremoved Event Listeners on short-lived objects.

```
  console.error(`Field error on ${err.field}: ${err.message}`);
  } else {
  throw new Error('System error occurred', { cause: err }); // Error chaining
  }
  } finally {
  console.log('Cleanup database connection.'); // Always executes!
  }
  }
  ```
```

The `finally` block executes unconditionally, even if `return` or `throw` is invoked inside `try` or `catch`.

---



**>>Solution Question** Java Script Internationalization API (Partial) Application: explain their theoretical foundations Calculus.
```

Generic Currying Implementation:

```javascript
function curry(fn) {
return function curried(...args) {
if (args.length >= fn.length) {
return fn.apply(this, args);
} else {
return function(...nextArgs) {
return curried.apply(this, args.concat(nextArgs));
};
}
};
}

const sum = (a, b, c) => a + b + c;
const curriedSum = curry(sum);
console.log(curriedSum(1)(2)(3)); // 6
console.log(curriedSum(1, 2)(3)); // 6
```

Benefits: Facilitates high modularity, function reusability, and configurable utility functions.

[Detailed Step-by-Step Answer]:
Here is a complete recursive deep clone algorithm handling edge cases:

```javascript
function deepClone(obj, hash = new WeakMap()) {
if (obj === null || typeof obj !== 'object') return obj;
if (obj instanceof Date) return new Date(obj.getTime());
if (obj instanceof RegExp) return new RegExp(obj);

// Handle circular references to prevent infinite recursion
if (hash.has(obj)) return hash.get(obj);

const copy = Array.isArray(obj) ? [] : Object.create(Object.getPrototypeOf(obj));
hash.set(obj, copy);

for (const key of Reflect.ownKeys(obj)) {
copy[key] = deepClone(obj[key], hash);
}
return copy;
}
```

>>Solution to Question 22: a custom implementation of the higher-order array method Array.prototype.myReduce() conforming to the ECMAScript specificati

```
  return accumulator;
  };
  ```
```

---



>>Solution Question Bank Java Script Debug System Tutorial Common Java Like Script explain ES6 Modules (node/react/front) applications.
```

- Output: Live bindings to memory references (updating exported variable updates all importing modules).

Mitigation: Explicitly set `massiveBuffer = null;` when heavy data structures are no longer required, allowing the Mark-and-Sweep garbage collector to reclaim heap allocation.
